

گزارش تمرین اول

داده کاوی - دکتر آبین

هدی عطاری – ۴۰۱۲۴۳۰۶۸

هدف از این تمرین، اعمال تکنیک‌های پیش‌پردازش و Feature Engineering بر روی مجموعه داده‌ی بانکی سپس آموزش یک مدل یادگیری ماشین با استفاده از ابزارهای AutoML برای پیش‌بینی متغیر هدف (y) بود. در این مسیر، تأثیر پیش‌پردازش بر عملکرد مدل نهایی نیز ارزیابی شد.

قسمت اول:

```
# TO-DO: fill 'categorical_cols' and fill na values in categorical columns of 'df' dataframe.

categorical_cols = [col for col in df.columns if df[col].dtype == 'object' and col != target_variable]

for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])

df.head()
```

(4) ✓ 0.1s Python

	age	job	marital	education	default	housing	loan	is_telephone_contact	month	day_of_week	duration	campaign	pdays	previous	poutcome	y
0	56	housemaid	married	basic4y	no	no	no	yes	may	mon	261	1	999	0	nonexistent	no
1	57	services	married	highschool	no	no	no	yes	may	mon	149	1	999	0	nonexistent	no
2	37	services	married	highschool	no	yes	no	yes	may	mon	226	1	999	0	nonexistent	no
3	40	admin.	married	basic6y	no	no	no	yes	may	mon	151	1	999	0	nonexistent	no
4	56	services	married	highschool	no	no	yes	yes	may	mon	307	1	999	0	nonexistent	no

قسمت دوم :

در اولین مرحله از پیش‌پردازش، برای مدیریت بهتر داده‌ها، ستون‌های دیتافریم بر اساس نوع ذاتی آن‌ها به چهار دسته (لیست) مجزا تقسیم شدند:

- متغیرهای پیوسته (**continuous_cols**): ستون‌هایی که مقادیر عددی پیوسته دارند شامل: ['age', 'duration', 'campaign', 'pdays', 'previous', 'poutcome']
- متغیرهای دودویی (**binary_cols**): ستون‌هایی که تنها دو حالت ممکن دارند شامل: ['default', 'housing', 'loan', 'is_telephone_contact']
- متغیرهای اسمی (**nominal_cols**): ستون‌های دسته‌ای که ترتیب خاصی بین مقادیر آن‌ها وجود ندارد شامل: ['job', 'marital', 'month', 'day_of_week', 'poutcome']
- متغیرهای ترتیبی (**ordinal_cols**): ستون‌های دسته‌ای که مقادیر آن‌ها دارای یک ترتیب یا سلسله مراتب منطقی هستند شامل: ['education']

```
> ✓ 0.0s
continuous_cols = ['age', 'duration', 'campaign', 'pdays', 'previous']
binary_cols = ['default', 'housing', 'loan', 'is_telephone_contact']
nominal_cols = ['job', 'marital', 'month', 'day_of_week', 'poutcome']
ordinal_cols = ['education']

Generate + Code + Markdown Python
```

قسمت سوم :

در این مرحله، یک زیرمجموعه از داده‌ها به نام `binary_df` ساخته شد که تنها شامل ستون‌های دودویی بود.

- تبدیل انجام شده: از آنجایی که مقادیر این ستون‌ها به صورت متنی (yes/no) بودند، با استفاده از متد `map()` در پانداس، تمام مقادیر 'yes' به عدد ۱ و تمام مقادیر 'no' به عدد ۰ تبدیل شدند. این کار باعث شد تا ویژگی‌های دودویی برای مدل قابل فهم شوند.

```
# To-DO: create 'binary_df' and replace all 'object' values with numeric ones
binary_df = df[binary_cols].copy()

# Convert yes/no to 1/0
for col in ['default', 'housing', 'loan', 'is_telephone_contact']:
    binary_df[col] = binary_df[col].map({'yes': 1, 'no': 0})

binary_df.head()

✓ 0.0s
```

	default	housing	loan	is_telephone_contact
0	0	0	0	1
1	0	0	0	1
2	0	1	0	1
3	0	0	0	1
4	0	0	1	1

قسمت چهارم:

برای متغیرهای اسمی، از آنجایی که هیچ ترتیبی بین مقادیر آن‌ها وجود ندارد (مثلاً شغل 'admin' از شغل 'student' برتر یا کمتر نیست)، از تکنیک **One-Hot Encoding** استفاده شد.

- روش اجرا: با استفاده از تابع `pd.get_dummies()`، یک دیتافریم جدید به نام `nominal_df` ایجاد شد. به ازای هر مقدار یکتای موجود در ستون‌های اسمی، یک ستون جدید با مقادیر ۰ و ۱ ساخته شد. این روش از ایجاد هرگونه ارزش‌گذاری کاذب توسط مدل جلوگیری می‌کند.

```
nominal_df = pd.get_dummies(
    df[nominal_cols],
    prefix=nominal_cols,
    prefix_sep='_',
    drop_first=False,
    dtype=int
)

nominal_df.head()

✓ 0.0s Python
```

	job_admin.	job_blue-collar	job_entrepreneur	job_housemaid	job_management	job_retired	job_self-employed	job_services	job_student	job_technician	...	month_oct	month_sep	d
0	0	0	0	0	1	0	0	0	0	0	...	0	0	
1	0	0	0	0	0	0	0	1	0	0	...	0	0	
2	0	0	0	0	0	0	0	1	0	0	...	0	0	
3	1	0	0	0	0	0	0	0	0	0	...	0	0	
4	0	0	0	0	0	0	0	1	0	0	...	0	0	

5 rows × 32 columns

قسمت پنجم :

در ستون‌های ترتیبی مانند education (تحصیلات)، ترتیب مقادیر اهمیت دارد (مثلاً مدرک دانشگاهی بالاتر از دیپلم است). بنابراین، به جای One-Hot Encoding، از روش نگاشت سفارشی استفاده شد.

- روش اجرا: ابتدا یک دیکشنری به نام education_order تعریف شد که در آن به هر سطح از تحصیلات یک عدد صحیح اختصاص داده شد (از ۰ تا ۶). سپس با استفاده از متد map()، مقادیر متنی ستون تحصیلات در دیتافریم ordinal_df به این اعداد دقیق و دارای ترتیب جایگزین شدند.

```
# PART 5 - create 'ordinal_df' according to above explanations

# Define the correct order for the ordinal column 'education'
education_order = {
    'unknown': -1,
    'illiterate': 0,
    'basic.4y': 1,
    'basic.6y': 2,
    'basic.9y': 3,
    'high.school': 4,
    'professional.course': 5,
    'university.degree': 6
}

# Create ordinal_df containing only ordinal columns
ordinal_df = df[ordinal_cols].copy()

# Map textual values to numeric order
ordinal_df['education'] = ordinal_df['education'].map(education_order)

ordinal_df.head()
```

مدل سازی و نتایج: از ۵۱٪ به ۶۲٪ رسیدیم.

```
Initialize Reactive Jupyter | Sync all State code
target_variable = 'y'
new_df = df[continuous_cols].join([binary_df, nominal_df, ordinal_df])

model = AutoML(task='classification', time_budget=120, verbose=0, estimator=sklearn.linear_model.LogisticRegression)
model.fit(new_df, df[target_variable])

df_test = new_df.join(df.y).sample(frac=0.3, random_state=313)
y_pred = model.predict(df_test.drop(target_variable, axis=1))

f1score = f1_score(df_test[target_variable], y_pred, pos_label='yes')
print(f'performance of model is {f1score} %')
```

[11] ✓ 2m 18.8s

... performance of model is 62.80388978930308 %

```
import zipfile
import joblib

joblib.dump(categorical_cols, "categorical_cols")
joblib.dump(continuous_cols, "continuous_cols")
joblib.dump(binary_cols, "binary_cols")
joblib.dump(nominal_cols, "nominal_cols")
joblib.dump(ordinal_cols, "ordinal_cols")

binary_df.to_csv('binary_df.csv', index=False)
nominal_df.to_csv('nominal_df.csv', index=False)
ordinal_df.to_csv('ordinal_df.csv', index=False)

df.to_csv('df.csv', index=False)

def compress(file_names):
    print("File Paths:")
    print(file_names)
    compression = zipfile.ZIP_DEFLATED
    with zipfile.ZipFile("result.zip", mode="w") as zf:
        for file_name in file_names:
            zf.write('./' + file_name, file_name, compress_type=compression)

file_names = ["categorical_cols", "continuous_cols", "binary_cols", "nominal_cols", "ordinal_cols"]
compress(file_names)
```

File Paths:
categorical_cols, continuous_cols, binary_cols, nominal_cols, ordinal_cols